

מדריך חינוך

לגנוב מהמקצוענים

הקובץ שאיתו עובד מי שבנה את Claude Code

6 העקרונות של בוריס צ'רני (ראש Claude Code) – ולצידם התבנית שלי בעברית, מותאמת גם למי שבא בלי רקע טכני.



■ מי ולמה

מי שבנה את הכלי – עובד ככה

בוריס צ'רני הוא היוצר וראש Claude Code ב-Antropic (בעברו מהנדס ראשי באינסטגרם, ומחבר הספר Programming TypeScript). אלה העקרונות שהוא עובד לפיהם – ולצידם התבנית בעברית שאני עצמי משתמש בה, גם למי שבא בלי רקע טכני.

הרעיון בשורה אחת

אותם עקרונות שמריצים את מי שבנה את Claude Code – מתורגמים לעברית פשוטה, לכל אחד.

איך קוראים את המדריך

- 1 קראו את העיקרון (אנגלית) ואת ההתאמה שלי (עברית).
- 2 העתיקו את מה שמתאים לכם לתוך קובץ CLAUDE.md שלכם.
- 3 טיפ של בוריס: בתוך Claude Code, הסימן # מוסיף כלל לקובץ בלחיצה אחת.



עובד גם אם אתם לא מתכנתים. CLAUDE.md הוא קובץ טקסט פשוט – כותבים בו פעם אחת, והוא נטען לבד בכל שיחה.

■ כלל הזהב

מה הופך CLAUDE.md למנצח

ארבעה כללים רשמיים מ-Anthropic. תכניסו את כולם – והקובץ באמת יעבוד.

1 קצר

שמרו אותו קצר וקריא. קובץ נפוח גורם ל-Claude ל-200 שורות. מתחת

2 הדגשה

רוצים שיציית? הוסיפו IMPORTANT או YOU MUST. Anthropic רשמית.

3 התחלה

הפקודה init/ יוצרת קובץ ראשוני מהפרויקט שלכם. משם רק משפרים.

4 כמו קוד

כשמשהו משתבש – פותחים את הקובץ, גוזמים מיותר, ובודקים שההתנהגות השתנתה.

דוגמה – שורות מתוך CLAUDE.md

🔖 העתיקו

PROMPT ●●●

CLAUDE.md

IMPORTANT: always reply in Hebrew.

Run: npm run build

Style: short, concrete, no fluff.

■ לפני / אחרי

שורה אחת – לפני ואחרי

אותה כוונה, ניסוח אחר. ככה שורה ב-CLAUDE.md הופכת מ"סתם טקסט" למשהו ש-Claude באמת מציית לו.

✗ "בבקשה תענה יפה ומקצועי."

✓ IMPORTANT: ענה בעברית, קצר, בלי הקדמות ובלי סופרלטיבים.

✗ "אתה יודע מה אני צריך."

✓ אני [התפקיד שלי]. הקהל שלי [מי]. המטרה [מה] – ענה בהתאם.

✗ "אל תחזור על הטעות הזאת."

✓ כל תיקון שלי → שורה חדשה כאן ככלל קבוע. (או לחצו # ב-Claude Code).

■ העקרונות • 1-3

אותם עקרונות, שתי שפות

כל עיקרון פעמיים: כפי שבוריס עובד (אנגלית), וכפי שאני מיישם (עברית).

Plan first: enter plan mode for any non-trivial task (3+ steps).

BORIS • EN

מתכננים לפני שמבצעים.

גל • HE

לפני משימה עם כמה שלבים – קודם תוכנית, ורק אז ביצוע. חוסך המון תיקונים בהמשך.

Use subagents liberally to keep the main context clean.

BORIS • EN

מפצלים עבודה ל"סוכני-משנה".

גל • HE

מחקר וחקירה עוברים הצידה – השיחה הראשית נשארת ממוקדת ונקייה.

Self-improvement loop: after any correction, write a rule so it never repeats.

BORIS • EN

כל תיקון הופך לכלל קבוע.

גל • HE

כל פעם שאתם מתקנים את ה-AI – כותבים את זה ל-CLAUDE.md. זו בדיוק העבודה של הקובץ.

■ העקרונות • 4-6

העקרונות – המשך

וזה נמשך: שלושת העקרונות האחרונים, שוב באנגלית ובעברית.

Verification before done: never mark a task complete without proving it works.

BORIS • EN

תמיד מוכיחים שזה עובד. גל • HE

לא סומכים על "נראה טוב". בודקים בפועל לפני שאומרים "סיימתי".

Demand elegance: for non-trivial changes, ask "is there a simpler way?"

BORIS • EN

מעדיפים את הדרך הפשוטה. גל • HE

הפתרון הפשוט כמעט תמיד הנכון. לא מסבכים בלי צורך.

Autonomous bug fixing: given a bug, just fix it – no hand-holding.

BORIS • EN

עצמאות בתיקון תקלות. גל • HE

מתארים את הבעיה ונותנים ל-AI לרוץ עד הפתרון – בלי לנהל כל צעד.

■ הקובץ • חלק 1

הקובץ של בוריס, כלשוננו

אלה עקרונות תזמור-העבודה (Workflow Orchestration) – להעתקה ישירה לתוך CLAUDE.md שלכם.

Workflow Orchestration

1. Plan by Default

Enter plan mode for ANY non-trivial task (3+ steps).
If something goes sideways, STOP and re-plan.

2. Subagent Strategy

Use subagents liberally to keep context clean.
One task per subagent, focused execution.

3. Self-Improvement Loop

After ANY correction, write a rule to prevent a repeat.
Review your lessons at session start.

4. Verification Before Done

Never mark done without proving it works.
Ask: "Would a staff engineer approve this?"

5. Demand Elegance (Balanced)

For non-trivial changes, seek the simpler way.
Skip it for obvious fixes – don't over-engineer.

6. Autonomous Bug Fixing

Given a bug report, just fix it – no hand-holding.
Point at logs, errors, failing tests – then resolve them.

■ הקובץ • חלק 2

ניהול משימות ועקרונות ליבה

החלק השני של הקובץ: איך לנהל משימה, ושלושת עקרונות-הליבה.

Task Management

Plan first: write the plan with checkable items.

Verify the plan before implementation.

Track progress; mark items complete as you go.

Explain changes with a high-level summary.

Capture lessons after each correction.

Core Principles

Simplicity first: make every change as simple as possible.

No laziness: find root causes, no temporary fixes.

Minimal impact: touch only what's necessary.

■ התבנית שלי

CLAUDE.md בעברית – להעתקה

התבנית שאני עובד איתה, מותאמת בעברית. העתיקו, מלאו את מה שבסוגריים [], והדביקו בקובץ שלכם.

CLAUDE.md #**## מי אני ומה חשוב לי**

ענה תמיד בעברית, קצר וברור.

אני **[התפקיד/התחום שלי]**. הקהל שלי: **[מי]**.**IMPORTANT:** אל תמציא עובדות. לא בטוח? עצור ושאל.**## איך אני אוהב לעבוד**

לפני משימה גדולה – קודם הצג תוכנית, ואז בצע.

התשובה הראשונה היא טיוטה. הצע לחדד ולקצר.

פרק משימות מורכבות לצעדים.

כללים קבועים (המקום ללקחים)**[כלל שלמדת: "כשאני אומר X, הכוונה ל-Y"]****YOU MUST:** **[הדבר הקריטי שאסור לפספס]**.

■ מתחילים

5 הדקות הראשונות

לא צריך להיות מתכנתים. ככה מתחילים היום בלי להסתבך – ועד סוף הדקה החמישית כבר יש לכם קובץ חי.

המטרה

CLAUDE.md ראשוני שנטען לבד בכל שיחה – כך Claude "מכיר אתכם" מהשנייה הראשונה.

צעד אחר צעד

- 1 הריצו init/ – נוצר CLAUDE.md ראשוני מהפרויקט שלכם.
- 2 הוסיפו 3 שורות: מי אתם, איך לדבר, ומה IMPORTANT.
- 3 בפעם הבאה שתתקנו את Claude – לחצו # כדי לשמור את זה לתמיד.



אין לכם Claude Code? אותו רעיון עובד גם ב-ChatGPT: פתחו Project או הנחיה קבועה, והדביקו שם את אותן שורות.

עכשיו תורכם.

בחרו עיקרון אחד, כתבו אותו לתוך CLAUDE.md שלכם, ותראו את ההבדל כבר בשיחה הבאה. אותם כללים שמריצים את מי שבנה את הכלי – עכשיו אצלכם.

כותבים פעם אחת, ו-Claude יודע לתמיד.



עוד מדריכי AI בעברית – כל שבוע ב-vaknin.ai. עקבו כדי לא לפספס את הבא.